

PRAKTIKUM SISTEM OPERASI

**MODUL 7
(SEMAPHORE)**



Oleh:

(Reyner Atira Prasetyo)

(2311104057)

**PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM
2026**

JURNAL 7

1. Buatlah 3 buah proses yaitu P1, P2 dan P3. P1 selalu menampilkan “kwak”, P2 selalu menampilkan “kwik”, P3 selalu menampilkan “kwek”.

The screenshot displays a Debian 8 desktop environment. At the top, a window titled "gedit" shows the contents of a file named "kwak_kwek.c". The code is written in C and implements a semaphore-based process synchronization. It defines a semaphore "Xinu" and three processes: P1, P2, and P3. P1 and P2 are resumed, while P3 is not. The code uses "sem_wait" and "sem_post" to manage the semaphore.

```

signal(sem_kwek);
}
return OK;
}

process P3(void) {
while(1) {
wait(sem_kwek);
printf("Kwek\n");
sleep(1000);
signal(sem_kwek);
}
return OK;
}

process main(void) {
// inisialisasi semaphore Xinu
sem_kwak = semcreate(1);
sem_kwik = semcreate(0);
sem_kwek = semcreate(0);

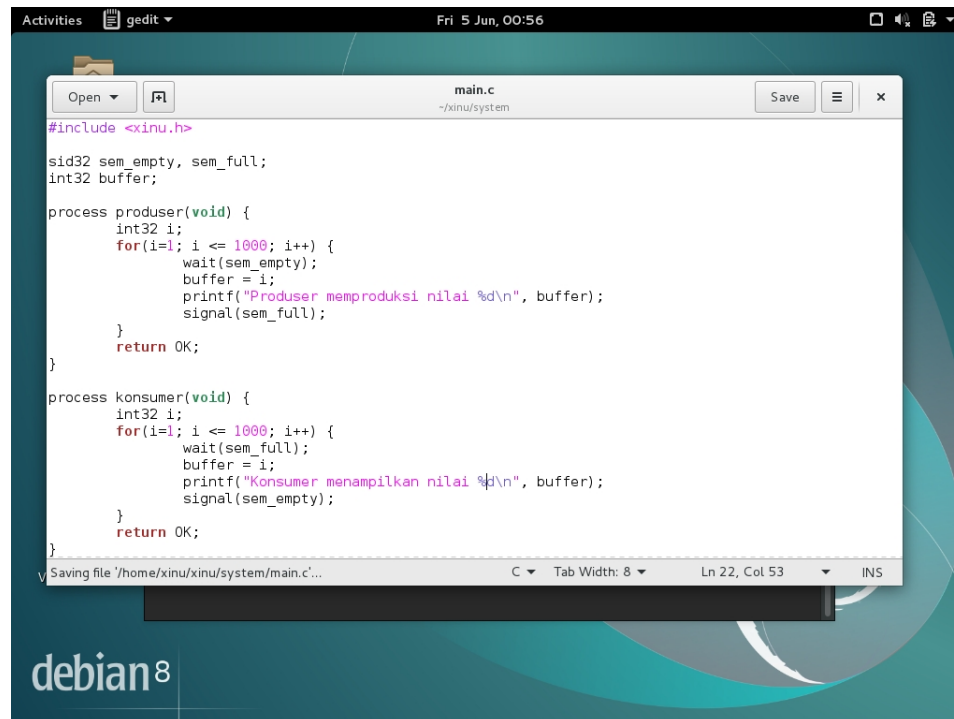
// membuat dan menjalankan proses di Xinu
resume(create(P1, 1024, 20, "Process P1", 0));
resume(create(P2, 1024, 20, "Process P2", 0));
resume(create(P3, 1024, 20, "Process P3", 0));
}

```

Below the code editor, a terminal window is open, showing the command prompt "xinu@reyner-2311104057: ~/xinu/compile". The terminal displays a list of files and directories, including "Kwik", "Kwek", "Kwak", "prev-version", "README", "xinu", and "VBox_GAs_6.1.5".

Program ini mengimplementasikan eksekusi sekuensial dari tiga proses konkuren menggunakan tiga buah semaphore sebagai mekanisme signaling antar proses. Semaphore pertama diinisialisasi dengan nilai 1 sebagai pemicu awal, sedangkan dua lainnya bernilai 0. Setiap proses akan menahan eksekusinya menggunakan fungsi wait() pada semaphore masing-masing, mencetak teks, lalu memberikan giliran ke proses selanjutnya menggunakan fungsi signal(). Untuk menyesuaikan dengan lingkungan Xinu dan mencegah sistem freeze, fungsi delay bawaan OS diganti dengan busy-waiting manual, dan ditambahkan infinite loop pada fungsi main() agar scheduler memiliki waktu untuk terus mengeksekusi ketiga proses tersebut.

2. Buatlah proses bernama produser yang memproduksi bilangan 1-1000. Buatlah proses bernama konsumen yang akan menampilkan nilai yang diproduksi oleh produser. Gunakan semaphore!

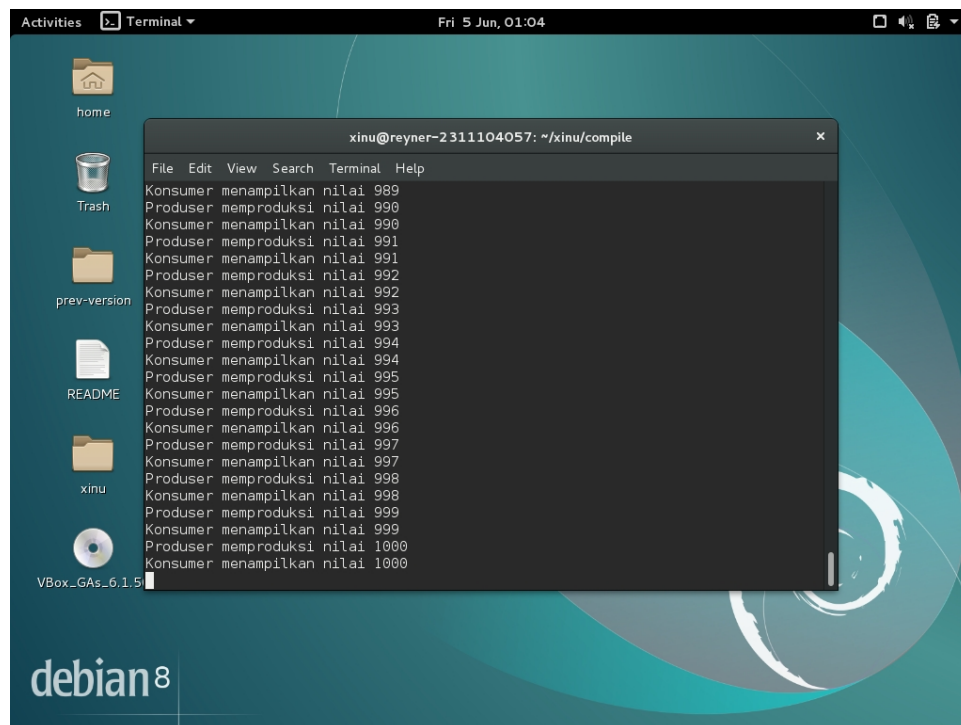


```
#include <xinu.h>

sid32 sem_empty, sem_full;
int32 buffer;

process produser(void) {
    int32 i;
    for(i=1; i <= 1000; i++) {
        wait(sem_empty);
        buffer = i;
        printf("Produser memproduksi nilai %d\n", buffer);
        signal(sem_full);
    }
    return OK;
}

process konsumer(void) {
    int32 i;
    for(i=1; i <= 1000; i++) {
        wait(sem_full);
        buffer = i;
        printf("Konsumer menampilkan nilai %d\n", buffer);
        signal(sem_empty);
    }
    return OK;
}
```



Program ini mensimulasikan masalah sinkronisasi Producer-Consumer menggunakan variabel buffer dan dua buah semaphore, yaitu `sem_empty` (bernilai 1) dan `sem_full` (bernilai 0). Proses produser menggunakan fungsi `wait(sem_empty)` untuk memastikan buffer kosong sebelum mengisi angka 1 hingga 1000, kemudian memberikan `signal(sem_full)` sebagai penanda data siap dibaca. Sebaliknya, proses konsumen memanggil `wait(sem_full)` untuk menunggu ketersediaan data, menampilkannya ke layar, lalu memanggil `signal(sem_empty)` untuk mengosongkan buffer kembali. Mekanisme Mutual Exclusion ini secara efektif mencegah penimpaan data yang belum dibaca dan memastikan tidak ada pembacaan data ganda.